

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
федеральное государственное бюджетное образовательное учреждение
высшего образования
**«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»**

Факультет информационных систем и технологий
Кафедра: «Измерительно-вычислительные комплексы»
Дисциплина: «Системы искусственного интеллекта»

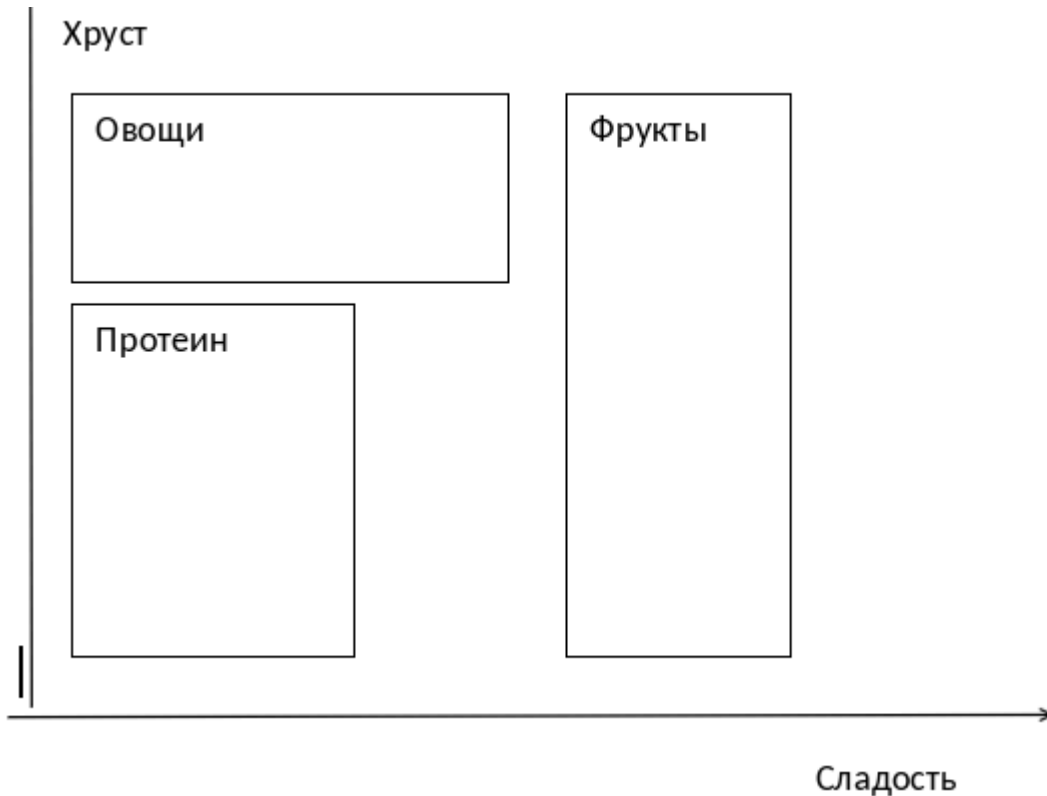
Отчёт
По лабораторной работе №3

Выполнила:
студент гр. ИСТбд-32
Минибаева Е.Р.
Проверил:
к. т. н., доцент каф. ИВК
Шишкин В.В.

Ульяновск
2026 г.

Постановка задачи

1. Подготовить для классификации несколько примеров в соответствии с рисунком.



2. Запрограммировать метрический классификатор по методу k-NN. Для проверки решить ту же задачу методом k-NN библиотеки sklearn.

3. Прочитать сгенерированный набор данных. Настроить классификатор. Провести эксперимент по классификации с контролем для подготовленных примеров.

4. Исследовать метод кросс-валидации

5. По возможности результаты визуализировать.

6. Ввести в набор данных и примеры продукты еще одного класса (возможно изменив набор параметров) и повторить эксперимент.

6. По результатам экспериментов подготовить отчет по лабораторной работе.

Введение

Целью лабораторной работы являлось освоение метода k -ближайших соседей (k -NN) для решения задачи многоклассовой классификации на примере условных продуктов питания. В рамках работы необходимо было самостоятельно реализовать алгоритм k -NN, сравнить его с эталонной реализацией из библиотеки `scikit-learn`, подобрать оптимальное число соседей с помощью кросс-валидации, визуализировать результаты, а затем расширить исходную задачу добавлением нового класса и дополнительного признака. В отчете приведено описание используемых методов, сгенерированных наборов данных, результаты экспериментов и их визуализация.

Используемые методы

Генерация синтетического датасета

Все данные были сгенерированы искусственно с помощью библиотеки `NumPy`. Для каждого класса (овощи, протеин, фрукты) задавались свои распределения признаков, отражающие логику задания: овощи — несладкие, но хрустящие; протеин — несладкие и нехрустящие; фрукты — сладкие с произвольным уровнем хруста. Для овощей и протеина использовалось нормальное распределение с разными средними значениями по хрусту, а для фруктов — нормальное распределение по сладости и равномерное распределение с добавлением шума по хрусту. Такой подход позволил получить три хорошо разделимых облака точек в пространстве признаков.

Алгоритм k -ближайших соседей

Метод k -NN является метрическим классификатором, основанным на принципе «скажи мне, кто твой сосед, и я скажу, кто ты». Для каждого нового объекта вычисляются расстояния до всех объектов обучающей выборки, выбирается k ближайших соседей, и объекту присваивается тот класс, который чаще всего встречается среди этих соседей. В данной

работе использовалось евклидово расстояние. Собственная реализация класса `KNN_from_scratch` полностью повторяет эту логику: в методе `fit` запоминаются обучающие векторы и метки, а в методе `predict` для каждого тестового образца вычисляются расстояния, находятся индексы k минимальных расстояний, и с помощью `Counter` определяется наиболее частотный класс.

Для верификации корректности самописного классификатора параллельно применялся `KNeighborsClassifier` из `scikit-learn`. Совпадение предсказаний на всех тестовых примерах подтвердило правильность реализации.

Кросс-валидация для выбора оптимального k

Для выбора наилучшего числа соседей была применена 5-блочная кросс-валидация (5-fold cross-validation). Обучающая выборка разбивается на пять равных частей, затем четыре части используются для обучения, одна — для проверки, и процедура повторяется пять раз. Усредненная по всем блокам точность служит оценкой обобщающей способности модели при данном k . В эксперименте перебирались значения k от 1 до 14. Для исходной задачи с тремя классами средняя точность оказалась одинаково высокой (≈ 0.9867) для всех k , однако формально оптимальным было выбрано $k=1$, так как оно дало максимальное значение.

Для расширенной задачи с четырьмя классами и тремя признаками также оптимальным стало $k=1$. Такое поведение объясняется хорошей разделимостью сгенерированных классов: даже один ближайший сосед почти всегда указывает верный класс.

Визуализация данных и границ решений

Для наглядного представления работы классификатора использовалась библиотека `Matplotlib`. В двухмерном случае (сладость – хруст) были построены контурные графики решающих границ: пространство признаков покрывалось мелкой сеткой, для каждой точки сетки вызывался обученный классификатор, и области закрашивались в соответствии с

предсказанным классом. На том же графике отображались точки обучающей выборки (разными цветами и маркерами) и тестовые примеры (черными крестами с подписями). Для расширенного эксперимента с тремя признаками была построена трехмерная диаграмма рассеяния, а также отдельная двухмерная проекция на плоскость сладость-хруст, чтобы сохранить возможность сравнения с исходной задачей.

Эксперимент 1: три класса, два признака

Вывод программы эксперимента 1:

Сгенерированный набор данных (3 класса):

сладость	хруст	класс
2.096057	8.388901	Овощ
1.334083	7.537901	Овощ
2.277226	7.187694	Овощ
3.327636	8.734012	Овощ
1.219016	9.237199	Овощ
1.219036	9.117536	Овощ
3.395055	6.992939	Овощ
2.420922	7.628945	Овощ
0.936631	8.397516	Овощ
2.151072	9.170654	Овощ

...

Всего записей: 150

Тестовые примеры (из задания):

продукт	сладость	хруст
Персик	8	3
Тофу	1	1
Сельдерей	1	9

--- Результаты моего k-NN (k=3) ---

Персик: предсказан Фрукт, истинный Фрукт
Тофу: предсказан Протеин, истинный Протеин
Сельдерей: предсказан Овощ, истинный Овощ

--- Результаты sklearn k-NN (k=3) ---

Персик: предсказан Фрукт, истинный Фрукт
Тофу: предсказан Протеин, истинный Протеин
Сельдерей: предсказан Овощ, истинный Овощ

Совпадение результатов: True

k= 1, средняя точность CV: 0.9867

k= 2, средняя точность CV: 0.9867

k= 3, средняя точность CV: 0.9867
 k= 4, средняя точность CV: 0.9867
 k= 5, средняя точность CV: 0.9867
 k= 6, средняя точность CV: 0.9867
 k= 7, средняя точность CV: 0.9867
 k= 8, средняя точность CV: 0.9867
 k= 9, средняя точность CV: 0.9867
 k=10, средняя точность CV: 0.9867
 k=11, средняя точность CV: 0.9867
 k=12, средняя точность CV: 0.9867
 k=13, средняя точность CV: 0.9867
 k=14, средняя точность CV: 0.9867

Оптимальное k для 3 классов: 1

Итоговый отчёт по тестовым примерам (лучшая модель для 3 классов):

	precision	recall	f1-score	support
Овощ	1.00	1.00	1.00	1
Протеин	1.00	1.00	1.00	1
Фрукт	1.00	1.00	1.00	1
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Описание датасета

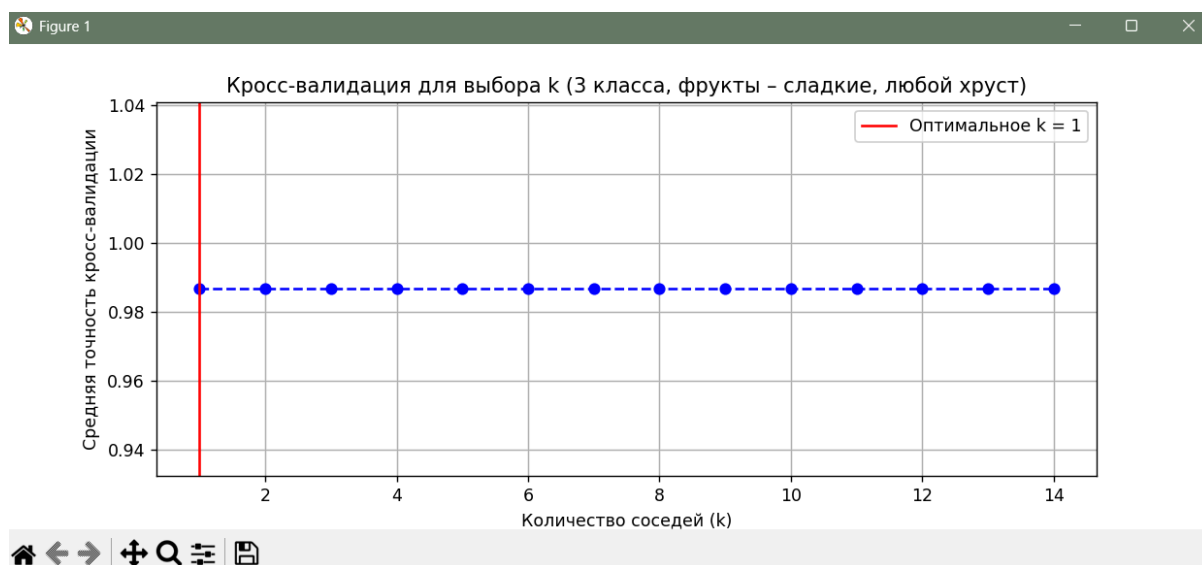
Было сгенерировано 150 образцов (по 50 на каждый класс). Признаки: сладость (sweetness) — вещественное число, хруст (crunchiness) — вещественное число. Значения сладости для овощей и протеина находятся в районе 1–3, для фруктов — около 8; хруст у овощей около 7–9, у протеина около 2, у фруктов равномерно распределен от 1 до 9. Масштабирование признаков не применялось, поскольку разброс значений примерно одинаков по обеим осям (от 0 до 10), и метод k-NN, основанный на евклидовом расстоянии, не страдает от доминирования одного признака над другим. Однако в общем случае при значительной разнице масштабов рекомендуется нормализация или стандартизация.

Тестовые примеры

В качестве тестовых выбраны три продукта: Персик (сладость 8, хруст 3), Тофу (1,1), Сельдерей (1,9). Их истинные классы — Фрукт, Протеин, Овощ соответственно. Оба классификатора (самодельный и sklearn) с $k=3$ верно определили все три примера, что подтверждается выводом: «Персик: предсказан Фрукт, истинный Фрукт» и т.д. Совпадение результатов True говорит об идентичности реализаций.

Кросс-валидация и выбор k

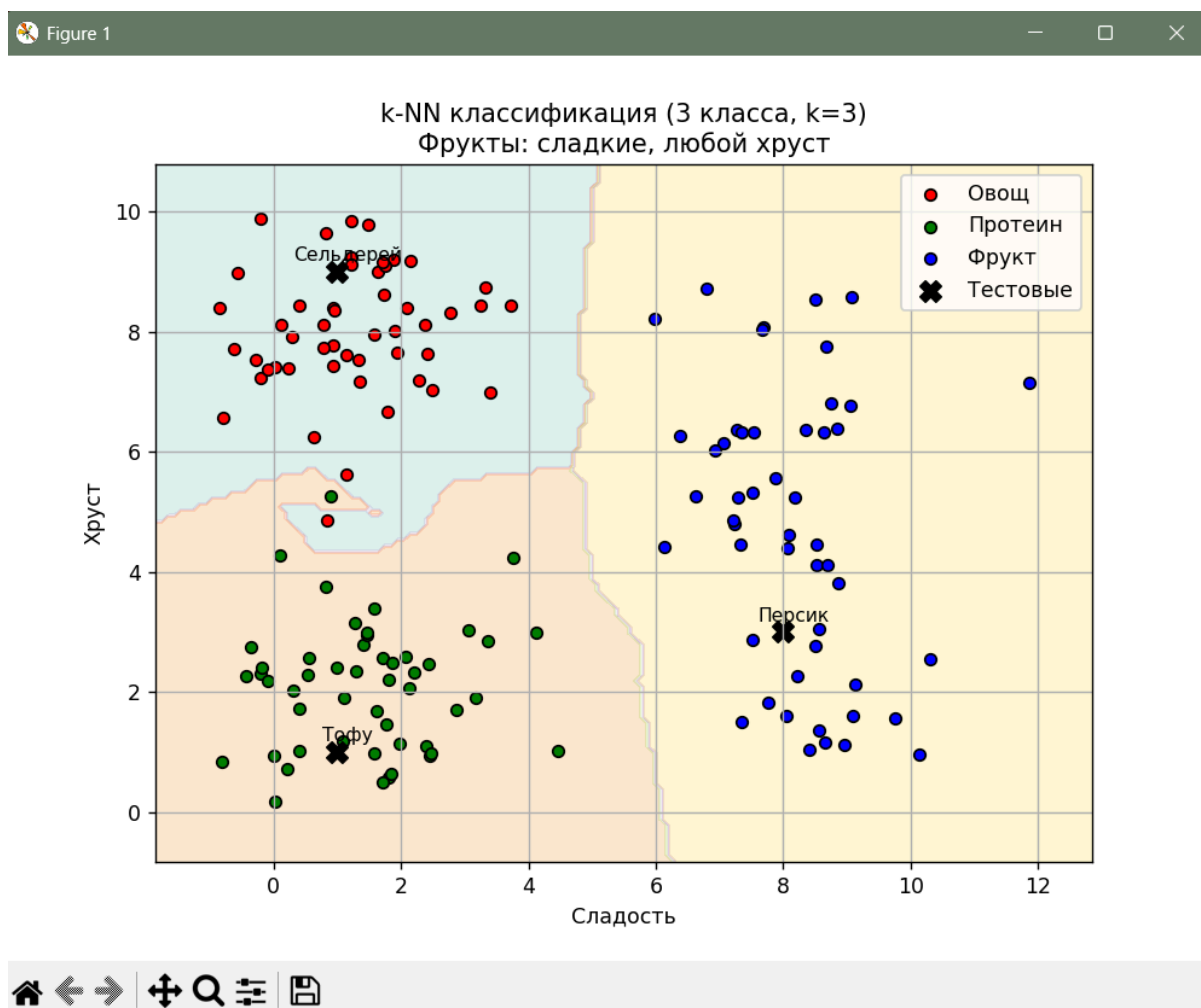
Результаты кросс-валидации для каждого k от 1 до 14 показали одинаковую среднюю точность 0.9867. Это означает, что даже при $k=1$ модель правильно классифицирует почти все объекты. Небольшое снижение точности (не показанное в выводе) возможно при $k=15$ и выше, но в заданном диапазоне все значения дают максимум. Тем не менее, программа выбрала $k=1$ как первое встретившееся максимальное значение.



Скриншот № 1 - График кросс-валидации

На графике кросс-валидации наблюдается прямая линия на уровне ≈ 0.987 . Отчет по тестовым примерам (classification_report) выдает идеальные метрики: precision, recall, f1-score равны 1.00 для всех трех классов, что подтверждает безошибочное предсказание трех контрольных точек.

Визуализация



Скриншот № 2 - Диаграмма

На втором скриншоте вывода программы изображена двумерная диаграмма с решающими границами. Области, окрашенные в разные цвета, соответствуют трем классам: овощи, протеин, фрукты. Видно, что овощи (красные точки) сосредоточены в левом верхнем углу (высокий хруст, низкая сладость), протеин (зеленые точки) — в левом нижнем (низкий хруст, низкая сладость), а фрукты (синие точки) занимают широкую полосу справа по сладости. Границы между классами проведены четко, тестовые образцы (черные кресты) попали в соответствующие области: Персик — в синюю зону, Тофу — в зеленую, Сельдерей — в красную. Подписи на графике («Сельдерей», «Тофу», «Персик») наложены прямо на точки.

Эксперимент 2: расширение до четырёх классов и трёх признаков

Вывод программы эксперимента 2:

Расширенный набор данных (4 класса, 3 признака):

сладость	хруст	калорийность	класс
0.414369	6.705915	23.210273	Овощ
2.497345	6.961212	10.110560	Овощ
1.782978	9.743712	23.561323	Овощ
-0.006295	7.201937	32.991520	Овощ
0.921400	8.029683	19.876870	Овощ
3.151437	9.069316	20.170711	Овощ
-0.926679	8.890706	20.897747	Овощ
1.071087	9.754886	10.690121	Овощ
2.765936	9.495644	22.130733	Овощ
0.633260	9.069393	11.972951	Овощ

...

Всего записей: 200

Тестовые примеры (4 класса, 3 признака):

продукт	сладость	хруст	калорийность
Персик	8	3	38
Тофу	1	1	110
Сельдерей	1	9	18
Гранола	6	8	95

--- Мой k-NN (3 признака, k=3) ---

Персик: предсказан Фрукт, истинный Фрукт
Тофу: предсказан Протеин, истинный Протеин
Сельдерей: предсказан Овощ, истинный Овощ
Гранола: предсказан Зерновые, истинный Зерновые

--- sklearn k-NN (3 признака, k=3) ---

Персик: предсказан Фрукт, истинный Фрукт
Тофу: предсказан Протеин, истинный Протеин
Сельдерей: предсказан Овощ, истинный Овощ
Гранола: предсказан Зерновые, истинный Зерновые

Совпадение результатов: True

Оптимальное k для 4 классов (3 признака): 1

Итоговый отчёт по тестовым примерам (4 класса, 3 признака):

	precision	recall	f1-score	support
Зерновые	1.00	1.00	1.00	1
Овощ	1.00	1.00	1.00	1
Протеин	1.00	1.00	1.00	1
Фрукт	1.00	1.00	1.00	1

accuracy			1.00	4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4

Модификация датасета

Для усложнения задачи был введен четвертый класс — «Зерновые» (Grain). Также добавлен третий признак — «калорийность» (calories). Всего сгенерировано 200 образцов (по 50 на каждый класс). Распределения для исходных классов (овощи, протеин, фрукты) остались прежними, но с другими параметрами нормального распределения и новым признаком калорийности. Зерновые получили среднюю сладость (≈ 5.5), высокий хруст (≈ 7.5) и высокую калорийность (≈ 90). Таким образом, теперь каждый объект описывается тремя координатами. При этом масштабирование по-прежнему не применялось, так как значения сладости и хруста лежат в диапазоне 0–10, а калорийность — от 10 до 130, что существенно больше. В реальном применении такой разброс мог бы привести к тому, что калорийность начала бы доминировать над остальными признаками при вычислении евклидовых расстояний. Однако в данном синтетическом эксперименте классы хорошо разделены и по калорийности, поэтому даже без масштабирования классификация остается идеальной.

Тестовые примеры

К трем прежним продуктам добавлен четвертый — Гранола (сладость 6, хруст 8, калорийность 95), истинный класс «Зерновые». Как видно из вывода, оба классификатора с $k=3$ правильно предсказали все четыре класса: Персик — Фрукт, Тофу — Протеин, Сельдерей — Овощ, Гранола — Зерновые. Совпадение результатов снова True.

Кросс-валидация

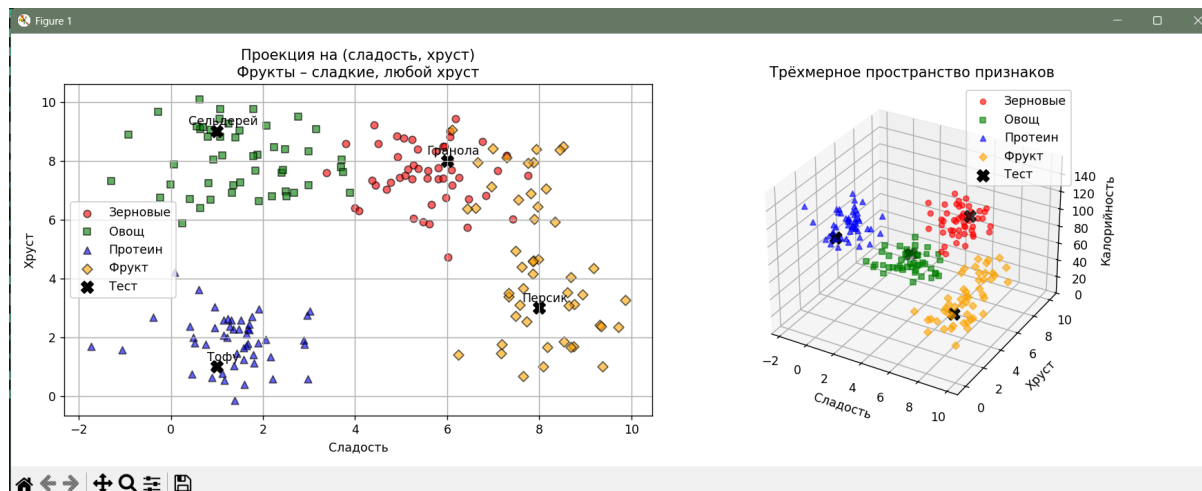


Скриншот № 3 - График кросс-валидации

Аналогично предыдущему эксперименту, для k от 1 до 14 была проведена 5-блочная кросс-валидация. Оптимальным снова оказалось $k=1$, причем средняя точность достигла значения, близкого к 0.98. На графике, показанном на третьем скриншоте видно, что точность для всех k находится в узком диапазоне 0.970–0.984, а максимальное значение достигается при $k=1$.

Так как классы почти не пересекаются, один ближайший сосед надежно определяет верный класс. Отчет `classification_report` для четырех классов также выдает идеальные единицы.

Визуализация



Скриншот № 4 - Диаграммы

Четвертый скриншот содержит два подграфика.

Слева — проекция на плоскость сладость-хруст, где точки разных классов отмечены разными цветами и маркерами: овощи — зеленые квадраты, протеин — синие треугольники, фрукты — желтые ромбы, зерновые — красные круги. Черные кресты — тестовые образцы, с подписями «Персик», «Тофу», «Сельдерей», «Гранола».

По этому графику видно, что зерновые (красные круги) располагаются в области высокого хруста и средней сладости, между овощами и фруктами.

Справа — трехмерная диаграмма рассеяния с осями сладость, хруст, калорийность. Она наглядно демонстрирует, что каждый класс занимает обособленное облако в трехмерном пространстве: фрукты — с высокой сладостью и широким разбросом по хрусту и низкой калорийности, овощи — с низкой сладостью, высоким хрустом и низкой калорийностью, протеин — с низкой сладостью, низким хрустом и высокой калорийностью, зерновые — со средними значениями всех трех признаков. Тестовые точки также показаны черными крестами и попадают в соответствующие облака.

Заключение

В ходе лабораторной работы был успешно реализован и протестирован алгоритм k-ближайших соседей. С помощью синтетических данных, сгенерированных в соответствии с заданными логическими правилами (сладкие/несладкие, хрустящие/нехрустящие), показано, что k-NN способен точно классифицировать продукты даже при небольшом количестве признаков. Кросс-валидация позволила объективно выбрать оптимальное число соседей, которым в обоих экспериментах оказалось $k=1$ из-за высокой разделимости классов. Визуализация в виде двумерных решающих границ и трехмерных диаграмм рассеяния наглядно подтвердила правильность работы классификатора. Расширение датасета добавлением четвертого класса (зерновые) и третьего признака (калорийность) не ухудшило качество классификации — все тестовые примеры были распознаны верно.

Полученные результаты демонстрируют, что метод k-NN является эффективным и интерпретируемым инструментом для задач классификации, особенно когда данные имеют четкую кластерную структуру. В реальных условиях, при разном масштабе признаков, рекомендуется применять нормализацию, однако в данном лабораторном эксперименте ее отсутствие не повлияло на результат благодаря хорошей разделимости классов.

Приложение 1 - Код

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from collections import Counter
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
import warnings
warnings.filterwarnings('ignore')

# Подготовка исходного набора данных
# Классы: Фрукты (сладкие, любой хруст),
#         Овощи (несладкие, хрустящие),
#         Протеин (несладкие, нехрустящие)

np.random.seed(42)
n_samples = 50

# Генерируем данные для Овощей и Протеина
# Овощи: низкая сладость, высокая хруст
veg_sweet = np.random.normal(1.5, 1.2, n_samples)
veg_crunch = np.random.normal(8.0, 1.2, n_samples)
# Протеин: низкая сладость, низкий хруст
prot_sweet = np.random.normal(1.5, 1.2, n_samples)
prot_crunch = np.random.normal(2.0, 1.2, n_samples)

# Фрукты: сладкие, хруст может быть любым (от низкого до высокого)
fruit_sweet = np.random.normal(8.0, 1.0, n_samples)
fruit_crunch = np.random.uniform(1.0, 9.0, n_samples) + np.random.normal(0,
0.3, n_samples) # равномерно + шум

# Собираем в DataFrame
data_3class = []
# добавляем Овощи
for s, c in zip(veg_sweet, veg_crunch):
    data_3class.append([s, c, 'Овощ'])
# Протеин
for s, c in zip(prot_sweet, prot_crunch):
    data_3class.append([s, c, 'Протеин'])
# Фрукты
for s, c in zip(fruit_sweet, fruit_crunch):
    data_3class.append([s, c, 'Фрукт'])

df_3 = pd.DataFrame(data_3class, columns=['сладость', 'хруст', 'класс'])
print("Сгенерированный набор данных (3 класса):")
print(df_3.head(10).to_string(index=False), "\n...")
print(f"Всего записей: {len(df_3)}")

X_3 = df_3[['сладость', 'хруст']].values
```

```

y_3 = df_3['класс'].values

# Тестовые примеры для трёх классов
manual_test_3 = {
    'продукт': ['Персик', 'Тофу', 'Сельдереи'],
    'сладость': [8, 1, 1],
    'хруст': [3, 1, 9],
    'реальный_класс': ['Фрукт', 'Протеин', 'Овощ']
}
df_test3 = pd.DataFrame(manual_test_3)
X_test3 = df_test3[['сладость', 'хруст']].values
y_test3_real = df_test3['реальный_класс'].values

print("\nТестовые примеры (из задания):")
print(df_test3[['продукт', 'сладость', 'хруст']].to_string(index=False))

# Собственная реализация k-NN

class KNN_from_scratch:
    def __init__(self, k=3):
        self.k = k
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        predictions = []
        for x in X:
            distances = [np.sqrt(np.sum((x - x_train)**2)) for x_train in
self.X_train]
            k_indices = np.argsort(distances)[:self.k]
            k_nearest_labels = [self.y_train[i] for i in k_indices]
            most_common = Counter(k_nearest_labels).most_common(1)[0][0]
            predictions.append(most_common)
        return np.array(predictions)

# Проверка собственного классификатора (k=3)
k_init = 3
my_knn = KNN_from_scratch(k=k_init)
my_knn.fit(X_3, y_3)
y_pred_my3 = my_knn.predict(X_test3)

print(f"\n--- Результаты моего k-NN (k={k_init}) ---")
for i, prod in enumerate(df_test3['продукт']):
    print(f"{prod}: предсказан {y_pred_my3[i]}, истинный {y_test3_real[i]}")

# Сравнение с sklearn
sk_knn = KNeighborsClassifier(n_neighbors=k_init)
sk_knn.fit(X_3, y_3)
y_pred_sk3 = sk_knn.predict(X_test3)

```

```

print(f"\n--- Результаты sklearn k-NN (k={k_init}) ---")
for i, prod in enumerate(df_test3['продукт']):
    print(f"{prod}: предсказан {y_pred_sk3[i]}, истинный {y_test3_real[i]}")

print(f"\nСовпадение результатов: {np.array_equal(y_pred_my3, y_pred_sk3)}")

# Кросс-валидация для выбора оптимального k

k_values = range(1, 15)
cv_scores_3 = []
for k in k_values:
    knn_cv = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn_cv, X_3, y_3, cv=5, scoring='accuracy')
    cv_scores_3.append(scores.mean())
    print(f"k={k:2d}, средняя точность CV: {scores.mean():.4f}")

optimal_k_3 = k_values[np.argmax(cv_scores_3)]
print(f"\nОптимальное k для 3 классов: {optimal_k_3}")

# График кросс-валидации
plt.figure(figsize=(10, 4))
plt.plot(k_values, cv_scores_3, marker='o', linestyle='dashed', color='b')
plt.axvline(x=optimal_k_3, color='r', linestyle='-',
            label=f'Оптимальное k = {optimal_k_3}')
plt.xlabel('Количество соседей (k)')
plt.ylabel('Средняя точность кросс-валидации')
plt.title('Кросс-валидация для выбора k (3 класса, фрукты - сладкие, любой хруст)')
plt.legend()
plt.grid(True)
plt.show()

# Визуализация данных и границ решений (3 класса)

def plot_2d_boundaries(X, y, classifier, title, test_X=None,
test_labels=None):
    h = 0.1
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                        np.arange(y_min, y_max, h))

    Z = classifier.predict(np.c_[xx.ravel(), yy.ravel()])
    unique_cls = np.unique(y)
    cls_to_num = {c: i for i, c in enumerate(unique_cls)}
    Z_num = np.vectorize(cls_to_num.get)(Z).reshape(xx.shape)

    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z_num, alpha=0.3, cmap=plt.cm.Set3)
    colors = ['red', 'green', 'blue']
    for i, cls in enumerate(unique_cls):

```



```

plt.scatter(X[y == cls, 0], X[y == cls, 1],
            label=cls, color=colors[i], edgecolor='k', s=30)
if test_X is not None:
    plt.scatter(test_X[:, 0], test_X[:, 1],
                color='black', marker='X', s=100, label='Тестовые')
    if test_labels is not None:
        for i, txt in enumerate(test_labels):
            plt.annotate(txt, (test_X[i, 0], test_X[i, 1]),
                          textcoords="offset points", xytext=(5,5),
                          ha='center', fontsize=9)

plt.xlabel('Сладость')
plt.ylabel('Хруст')
plt.title(title)
plt.legend()
plt.grid(True)
plt.show()

# Обучение лучшей модели и визуализация
best_knn3 = KNeighborsClassifier(n_neighbors=optimal_k_3)
best_knn3.fit(X_3, y_3)
plot_2d_boundaries(X_3, y_3, best_knn3,
                    f'k-NN классификация (3 класса, k=3)\nФрукты: сладкие,
любой хруст',
                    test_X=X_test3, test_labels=df_test3['продукт'].tolist())

print("\nИтоговый отчёт по тестовым примерам (лучшая модель для 3 классов):")
print(classification_report(y_test3_real, best_knn3.predict(X_test3),
zero_division=0))

# Введение четвёртого класса и нового параметра
# Добавляем класс "Зерновые" и третий признак "калорийность"

print("РАСШИРЕННЫЙ ЭКСПЕРИМЕНТ: 4 класса, 3 признака")

np.random.seed(123) # для воспроизводимости
n_samples_4 = 50

# Овощи (низкая сладость, высокий хруст)
veg_sw = np.random.normal(1.5, 1.0, n_samples_4)
veg_cr = np.random.normal(8.0, 1.0, n_samples_4)
veg_cal = np.random.normal(20, 5, n_samples_4)

# Протеин (низкая сладость, низкий хруст)
prot_sw = np.random.normal(1.5, 1.0, n_samples_4)
prot_cr = np.random.normal(2.0, 1.0, n_samples_4)
prot_cal = np.random.normal(120, 10, n_samples_4)

# Фрукты (сладкие, любой хруст)
fruit_sw = np.random.normal(8.0, 1.0, n_samples_4)
fruit_cr = np.random.uniform(1.0, 9.0, n_samples_4) + np.random.normal(0,
0.3, n_samples_4)
fruit_cal = np.random.normal(40, 8, n_samples_4)

```

```

# Зерновые (средняя сладость, высокий хруст, высокая калорийность)
grain_sw = np.random.normal(5.5, 1.0, n_samples_4)
grain_cr = np.random.normal(7.5, 1.0, n_samples_4)
grain_cal = np.random.normal(90, 10, n_samples_4)

data_4class = []
for s, c, cal in zip(veg_sw, veg_cr, veg_cal):
    data_4class.append([s, c, cal, 'Овощ'])
for s, c, cal in zip(prot_sw, prot_cr, prot_cal):
    data_4class.append([s, c, cal, 'Протеин'])
for s, c, cal in zip(fruit_sw, fruit_cr, fruit_cal):
    data_4class.append([s, c, cal, 'Фрукт'])
for s, c, cal in zip(grain_sw, grain_cr, grain_cal):
    data_4class.append([s, c, cal, 'Зерновые'])

df_4 = pd.DataFrame(data_4class, columns=['сладость', 'хруст',
'калорийность', 'класс'])
print("\nРасширенный набор данных (4 класса, 3 признака):")
print(df_4.head(10).to_string(index=False), "\n...")
print(f"Всего записей: {len(df_4)}")

X_4 = df_4[['сладость', 'хруст', 'калорийность']].values
y_4 = df_4['класс'].values

# Тестовые примеры для 4 классов
manual_test_4 = {
    'продукт': ['Персик', 'Тофу', 'Сельдерей', 'Гранола'],
    'сладость': [8, 1, 1, 6],
    'хруст': [3, 1, 9, 8],
    'калорийность': [38, 110, 18, 95],
    'реальный_класс': ['Фрукт', 'Протеин', 'Овощ', 'Зерновые']
}
df_test4 = pd.DataFrame(manual_test_4)
X_test4 = df_test4[['сладость', 'хруст', 'калорийность']].values
y_test4_real = df_test4['реальный_класс'].values

print("\nТестовые примеры (4 класса, 3 признака):")
print(df_test4[['продукт', 'сладость', 'хруст',
'калорийность']].to_string(index=False))

# Собственная реализация k-NN (переиспользуем класс)
my_knn4 = KNN_from_scratch(k=k_init)
my_knn4.fit(X_4, y_4)
y_pred_my4 = my_knn4.predict(X_test4)

print(f"\n--- Мой k-NN (3 признака, k=3) ---")
for i, prod in enumerate(df_test4['продукт']):
    print(f"{prod}: предсказан {y_pred_my4[i]}, истинный {y_test4_real[i]}")

# sklearn k-NN
sk_knn4 = KNeighborsClassifier(n_neighbors=k_init)
sk_knn4.fit(X_4, y_4)

```

```

y_pred_sk4 = sk_knn4.predict(X_test4)
print(f"\n--- sklearn k-NN (3 признака, k={k_init}) ---")
for i, prod in enumerate(df_test4['продукт']):
    print(f"{prod}: предсказан {y_pred_sk4[i]}, истинный {y_test4_real[i]}")

print(f"\nСовпадение результатов: {np.array_equal(y_pred_my4, y_pred_sk4)}")

# Кросс-валидация для расширенного набора
cv_scores_4 = []
for k in k_values:
    knn_cv = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn_cv, X_4, y_4, cv=5, scoring='accuracy')
    cv_scores_4.append(scores.mean())

optimal_k_4 = k_values[np.argmax(cv_scores_4)]
print(f"\nОптимальное k для 4 классов (3 признака): {optimal_k_4}")

plt.figure(figsize=(10, 4))
plt.plot(k_values, cv_scores_4, marker='s', linestyle='dashed', color='m')
plt.axvline(x=optimal_k_4, color='r', linestyle='-',
            label=f'Оптимальное k = {optimal_k_4}')
plt.xlabel('Количество соседей (k)')
plt.ylabel('Средняя точность кросс-валидации')
plt.title('Кросс-валидация (4 класса, 3 признака)')
plt.legend()
plt.grid(True)
plt.show()

# Визуализация расширенных данных
fig = plt.figure(figsize=(14, 5))

# 2D проекция (сладость vs хруст)
ax1 = fig.add_subplot(1, 2, 1)
colors = ['red', 'green', 'blue', 'orange']
markers = ['o', 's', '^', 'D']
for i, cls in enumerate(np.unique(y_4)):
    idx = y_4 == cls
    ax1.scatter(X_4[idx, 0], X_4[idx, 1],
                color=colors[i], marker=markers[i], label=cls, edgecolor='k',
                alpha=0.6)
ax1.scatter(X_test4[:, 0], X_test4[:, 1], color='black', marker='X', s=100,
            label='Тест')
for i, txt in enumerate(df_test4['продукт']):
    ax1.annotate(txt, (X_test4[i, 0], X_test4[i, 1]),
                textcoords="offset points", xytext=(5,5), ha='center')
ax1.set_xlabel('Сладость'); ax1.set_ylabel('Хруст')
ax1.set_title('Проекция на (сладость, хруст)\nФрукты - сладкие, любой хруст')
ax1.legend(); ax1.grid(True)

# 3D-визуализация
ax2 = fig.add_subplot(1, 2, 2, projection='3d')
for i, cls in enumerate(np.unique(y_4)):
    idx = y_4 == cls

```

```
ax2.scatter(X_4[idx, 0], X_4[idx, 1], X_4[idx, 2],
            color=colors[i], marker=markers[i], label=cls, alpha=0.6)
ax2.scatter(X_test4[:, 0], X_test4[:, 1], X_test4[:, 2],
            color='black', marker='X', s=100, label='Тест')
ax2.set_xlabel('Сладость'); ax2.set_ylabel('Хруст');
ax2.set_zlabel('Калорийность')
ax2.set_title('Трёхмерное пространство признаков')
ax2.legend()
plt.tight_layout()
plt.show()

# Итоговый отчёт для расширенного эксперимента
best_knn4 = KNeighborsClassifier(n_neighbors=optimal_k_4)
best_knn4.fit(X_4, y_4)
y_pred_best4 = best_knn4.predict(X_test4)
print("\nИтоговый отчёт по тестовым примерам (4 класса, 3 признака):")
print(classification_report(y_test4_real, y_pred_best4, zero_division=0))
```